The `before_validation` and `after_validation` calls also accept the `on: :create` or `on: :update` parameter, which will cause the callback to be called only on the selected operation.

In addition to these sixteen calls, the `after_find` callback is invoked after any find operation, and `after_initialize` is invoked after an Active Record model object is created.

To have your code execute during a callback, you need to write a handler and associate it with the appropriate callback.

We have two basic ways of implementing callbacks.

The preferred way to define a callback is to declare handlers. A handler can be either a method or a block. You associate a handler with a particular event using class methods named after the event. To associate a method, declare it as private or protected, and specify its name as a symbol to the handler declaration. To specify a block, simply add it after the declaration. This block receives the model object as a parameter:

```
class Order < ApplicationRecord
  before_validation :normalize_credit_card_number
  after_create do |order|
    logger.info "Order #{order.id} created"
  end
  protected
  def normalize_credit_card_number
    self.cc_number.gsub!(/[-\s]/, '')
  end
end
```

You can specify multiple handlers for the same callback. They will generally be invoked in the order they're specified unless a handler thows `:abort`, in which case the callback chain is broken early.

Alternately, you can define the callback instance methods using callback objects, inline methods (using a proc), or inline eval methods (using a string). See the online documentation for more details.[100]